Facultad de ingeniería

Manejo de archivos

Docente: ING. Rojas Vásquez Manuel Gustavo

Manual de usuario proyecto 2

Nombre: Davis José Alexis García López

Carne: 1554725

Quetzaltenango 9 de septiembre del 2026

Que hace el sistema

El sistema de **Agenda de Contactos - Acceso Aleatorio** es una herramienta de escritorio construida en Python 3.13 diseñada para el almacenamiento optimizado y gestión persistente de contactos. A diferencia de las bases de datos secuenciales simples, este programa utiliza estructuras físicas rígidas de **100 bytes** en disco con acceso aleatorio instantáneo `O(1)` mediante punteros directos.

Con este sistema el usuario puede:

- **Registrar y actualizar contactos:** Guardar nombres, teléfonos y correos electrónicos asignando un ID único de acceso.
- **Acceso directo por índice:** Ubicar y modificar registros in situ sin necesidad de recorrer el archivo completo.
- **Exportación multilenguaje y reportes:** Generar respaldos estructurados en JSON y reportes formateados en PDF mediante `fpdf2`.
- **Auditoría de Integridad (SHA-256):** Verificar visualmente si la base de datos ha sido alterada o dañada externamente.
- **Respaldo comprimido ZIP:** Crear copias atómicas con sello de fecha y hora para migración segura.

1. Requisitos de Instalación y Ejecución

> **Requisitos del Entorno**
>
> - Sistema Operativo: Windows 10 u 11.
> - Intérprete Python: Versión 3.13 o superior.
> - Librería Externa: `fpdf2` (para exportación PDF).

Pasos para la instalación y puesta en marcha:

1. Abra la terminal del sistema (PowerShell o CMD) en el directorio del proyecto.
2. Instale la librería necesaria ejecutando: `pip install fpdf2`.
3. Ejecute el punto de entrada principal con el comando: `python main.py`.

2. Estructura de Archivos de Datos

La persistencia se organiza automáticamente dentro de la carpeta `data/` en el directorio raíz:

Archivo	Formato	Descripción y Función
`data/contactos.dat`	Binario (100b)	Contiene los registros de contactos almacenados físicamente en bloques fijos de 100 bytes en codificación UTF-8.
`data/indice.json`	JSON Legible	Índice primario que vincula cada `ID` con el número de registro físico (offset de 100 bytes).
`data/contactos.sha256`	Texto Hash	Almacena la firma digest SHA-256 calculada sobre el archivo binario para control de integridad.

3. Guía Detallada de la Interfaz Gráfica

3.1 Pestaña "Contactos" (Formulario y Gestión CRUD)

Esta pestaña constituye el centro de operaciones. Permite ingresar nuevos contactos o seleccionar un registro de la tabla para su edición o eliminación.

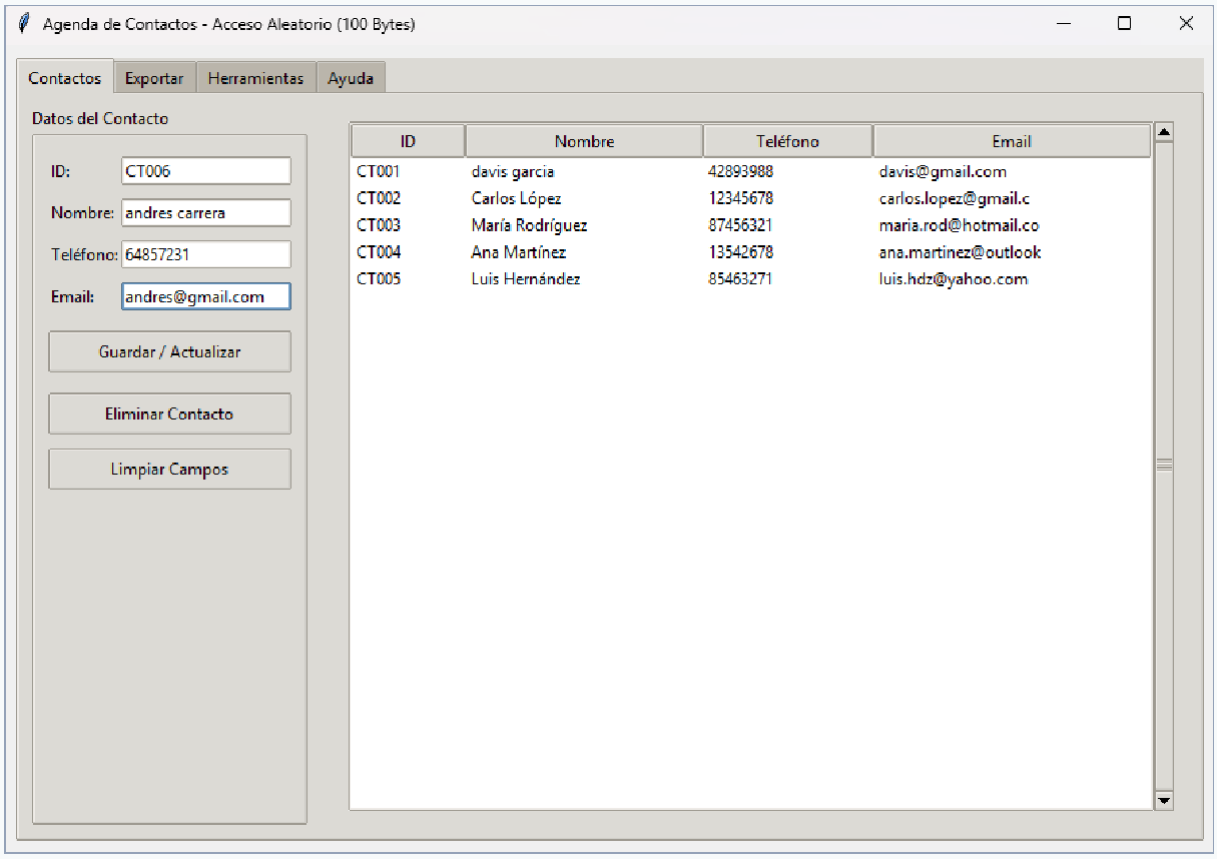


Figura 1. Formulario de la pestaña Contactos listo para guardar el registro CT006.

Al llenar los campos **ID**, **Nombre**, **Teléfono** y **Email** y hacer clic en `Guardar / Actualizar`, el sistema calcula la posición exacta del puntero binario.

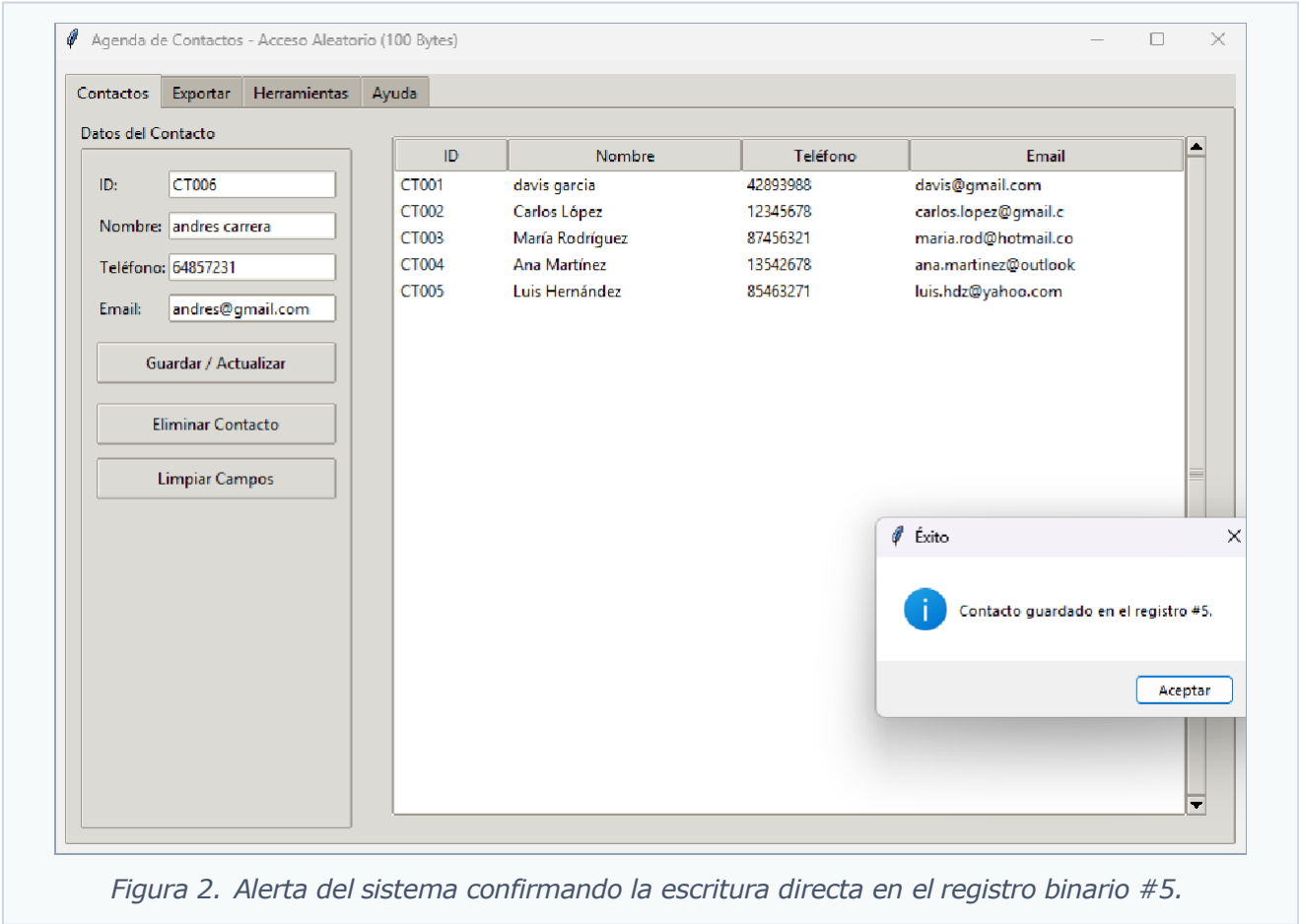


Figura 2. Alerta del sistema confirmando la escritura directa en el registro binario #5.

Tras aceptar la notificación de confirmación, la tabla `Treeview` refresca inmediatamente su contenido mostrando la totalidad de los registros guardados en la agenda.

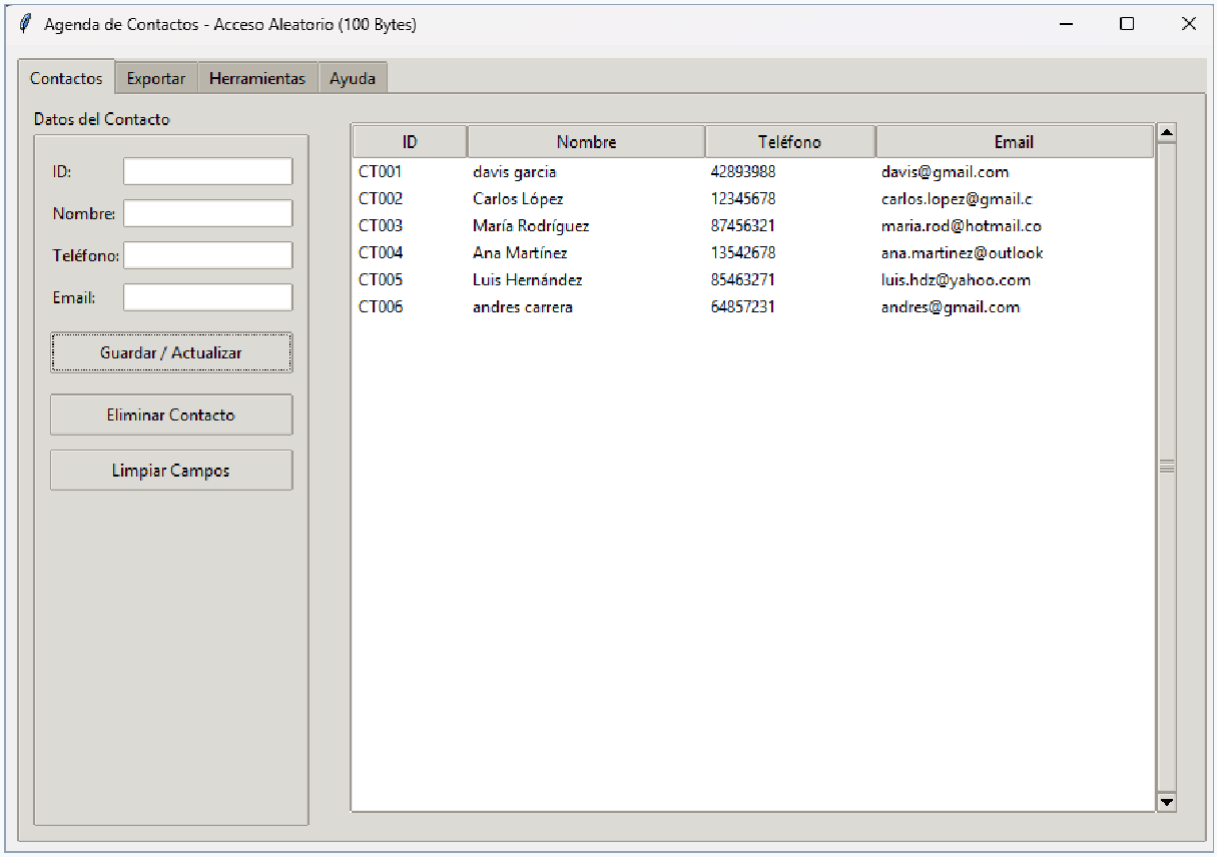


Figura 3. Vista de la agenda con los 6 registros integrados y alineados correctamente.

3.2 Pestaña "Exportar"

Permite la generación de archivos legibles y formalizados fuera del entorno binario del sistema.

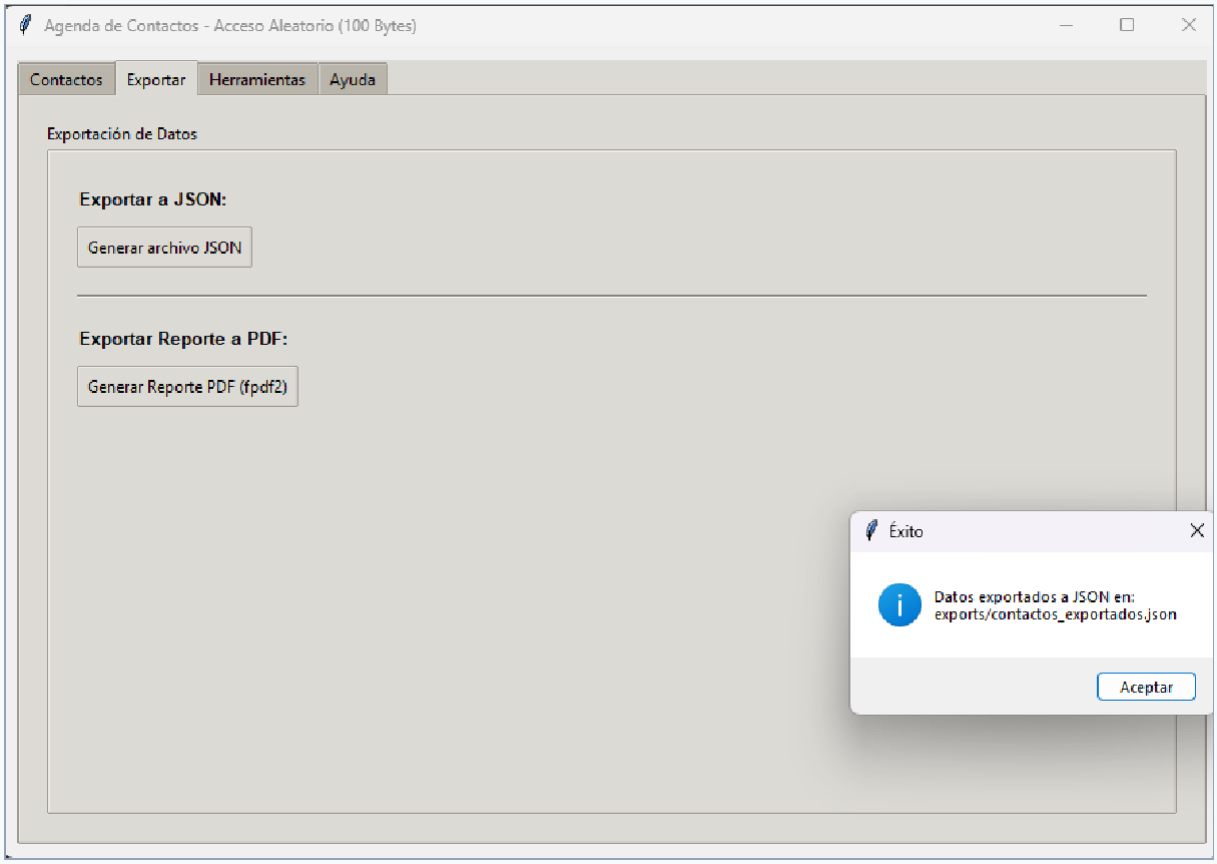


Figura 4. Generación del reporte dinámico en formato JSON dentro de la carpeta exports/.

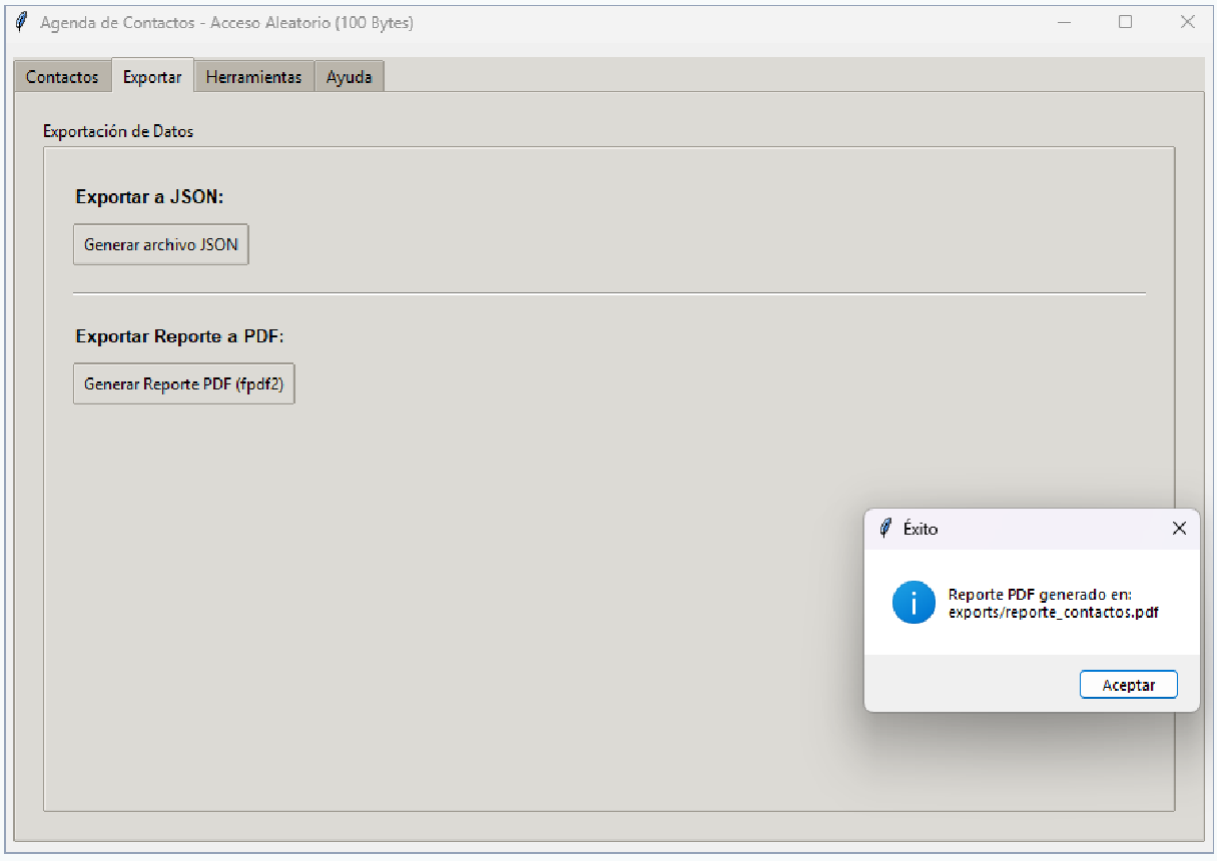


Figura 5. Notificación de éxito tras compilar el reporte imprimible PDF con fpdf2.

3.3 Pestaña "Herramientas"

Proporciona utilidades de auditoría de seguridad, backups comprimidos y consulta de metadatos del sistema operativo.

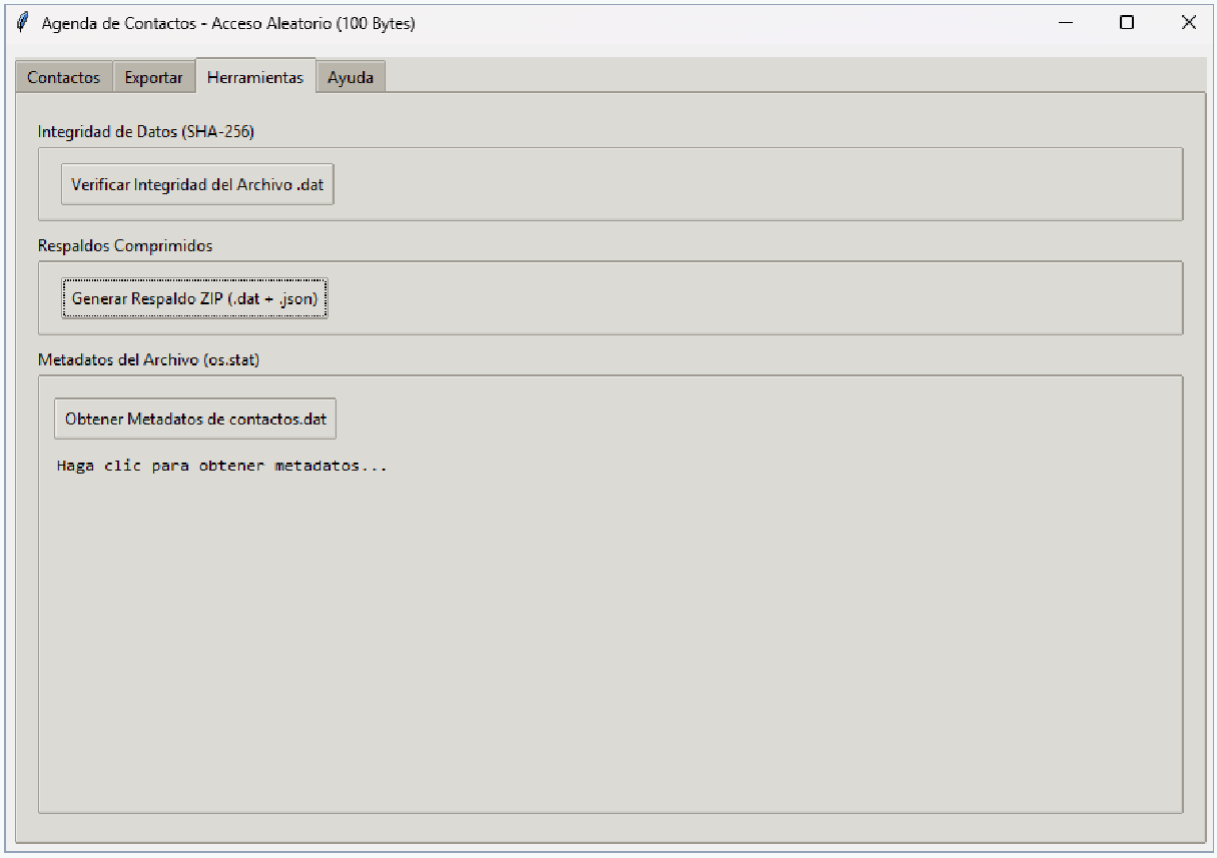


Figura 6. Pestaña de herramientas para verificación SHA-256 y compresión ZIP.

3.4 Pestaña "Ayuda"

Despliega el resumen técnico con las especificaciones del desarrollo y arquitectura de datos evaluada en la materia.

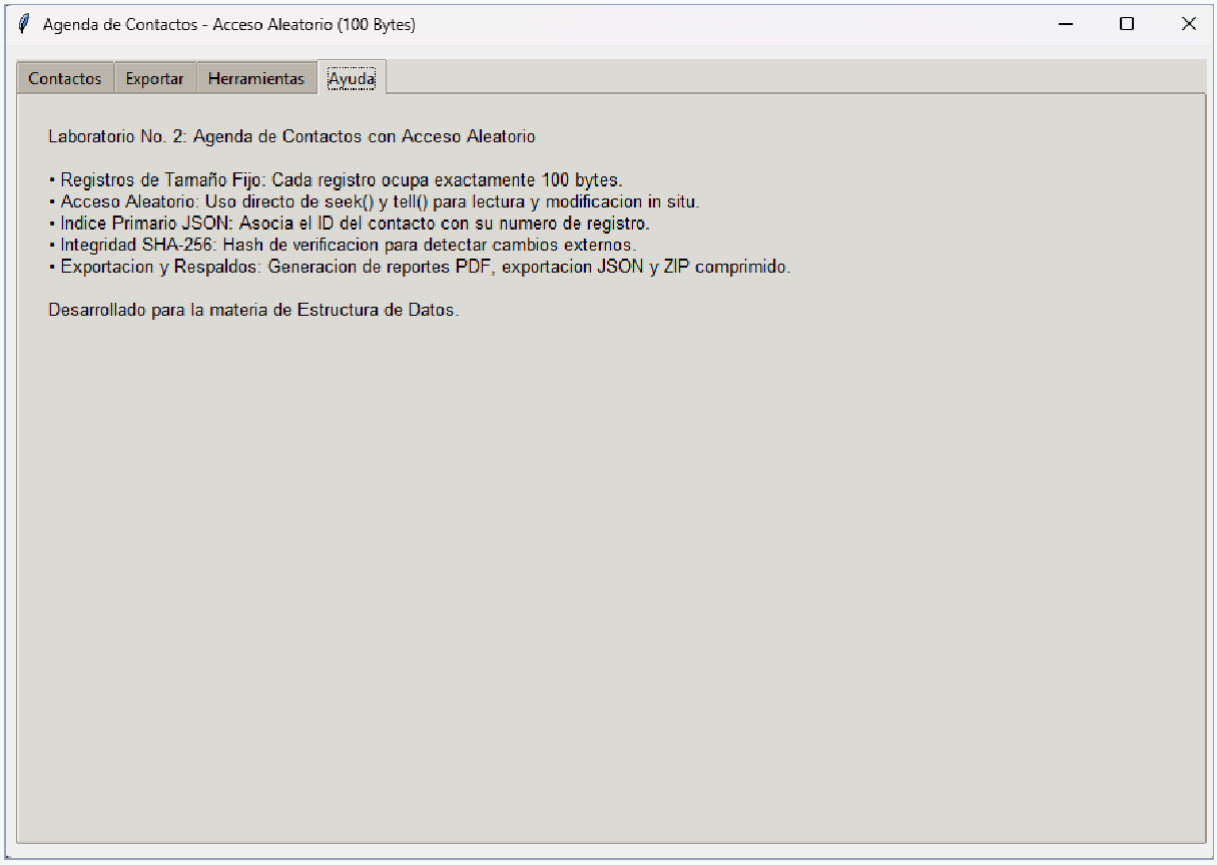


Figura 7. Pestaña de información técnica sobre la implementación de la agenda.

5. Preguntas Frecuentes

> **¿Por qué el archivo contactos.dat no tiene saltos de línea al abrirlo en VS Code?**
>
> Al ser un archivo binario puramente físico, los registros de 100 bytes se escriben en flujo continuo sin caracteres `\n`. El programa calcula la posición de lectura con `seek(num_registro * 100)`.